Test connaissance : Architecture multicores

1.1 – A quoi sert le pipeline ?

1.2 – Quel est le rôle d'une mémoire cache ?

1.3 – Quel temps de réponse de réponse est attendu pour le cache de niveau L1 ?

1.4 – Donner la définition d'un cache associatif.

1.5 – Définir la notion de cohérence de cache

1.6 – Expliquer les différentes politiques d'écriture entre le cache et la mémoire DRAM.

1.7 - Expliquer la différence entre une architecture superscalaire et VLIW

1.8 – Quels sont les cas d'étude qui défavorisent ce type d'architecture ?

1.9 – Expliquer la différence entre le parallélisme intracore et intercores. Qui est en charge de la gestion de ce parallélisme (OS, compilateur, utilisateur, …).

1.10 – Expliquer les différentes techniques de modélisation de la consommation pour un core de processeur

1.11 – Comment évolue la consommation en fonction du parallélisme intracore ?

1.12 - Expliquer la différence entre le DVFS et le DPM ?

1.13 - Considérons une architecture comportant N cores de processeur et des tâches périodiques définies par leur temps d'exécution Ci et leur période Pi. Quelle condition devra être satisfaite pour pouvoir exécuter ces tâches ?

Préciser pour chaque réponse, lorsque c'est pertinent, les gains attendus (performances ou consommation).

1.1 : le pipeline permet : de chevaucher l'exécution de plusieurs instructions.
chaque instruction se trouve dans un étage différent du pipeline
le gain recherché est une augmentation du débit d'instructions et de la fréquence de fonctionnement.
architecture pipeline → instruction par cycle.

1.2 : rôle d'une mémoire cache : le cache est une mémoire SRAM rapide située entre le processeur et la RAM.
CPU → L1 L2 L3 → RAM.
le cache permet de réduire le temps moyen d'accès aux instructions et aux données. Le gain attendu est une réduction du temps d'exécution en évitant le plus possible l'accès à la mémoire principale.

1.3 : temps de réponse attendu pour le cache L1 : Est de l'ordre de 1 ns, soit 1 à 10 cycle du CPU.

1.4 : Définition d'un cache associatif : permet à une donnée mémoire d'être placée dans plusieurs lignes possible du cache.
Cache direct, l'emplacement est imposé.
gain attendu → réduction des défauts de conflit
Voir politique de remplacement pour les caches associatifs :
LRU, FIFO, Aléatoire :
LRU → réduit le taux d'échec mais plus complexe.

1.5 : cohérence de cache : consiste à garantir la cohérence des différentes copies d'une même donnée présente dans le système mémoire. Lorsqu'une copie est modifiée, il faut s'assurer que le processeur ou d'autres éléments utilisant une copie obsolète

1.6 : Politique d'écriture / DRAM :
→ write through : Donnée est écrite dans le cache et dans la mémoire principale.
(mémoire principale et le cache possède donc toj une valeur identique)
→ write back : Donnée est d'abord modifiée dans le cache / remet la mémoire à jour qu'au remplacement d'une ligne modifiée.
→ write Buffer : Données à écrire sont temporairement stockées temporairement les écritures pour éviter de bloquer le processeur.

1.7 : différence entre le VLIW et le superscalaire : Le matériel détecte dynamiquement les instructions pouvant être exécuter en //.
compilateur. le matériel. Dans un VLIW, cette tâche est réalisée statiquement par le compilateur.

1.8 : Cas qui défavorise ce type architecture.
Pour processeur superscalaire les cas les plus défavorables sont les dépendances entre instructions or le parallélisme ne peut pas être utilisé et donc on doit attendre la fin des instructions.

1.9 : Parallélisme intracore / intercores et qui le gère :
intracore : // à l'intérieur d'un coeur.
Pour un type scalaire : gestion = matériel.
Pour un VLIW : gestion = compilateur.
intercores : // entre plusieurs coeur :
gestion = système d'exploitation (OS)
→ l'OS distribue les instructions / applications / tâches à exécuter en parallèle.

1.10 : Techniques de modélisation de la consommation :
→ architectural
→ utilisateur.

1.11. Comment évolue la consommation en fonction du parallélisme intra-cœur :
→ parallélisme intra-cœur ↗ ⇒ puissance / consommation instantanée ↗.

1.12 : Différence entre DVFS. DPM :

1.13 : Conditions pour la tâches périodiques sur N cœur.
défini par chaque tâche : $C_i = WCET$. | charge : $u_i = \frac{C_i}{P_i}$
P_i = période

pour EDF sur un cœur : $u = \sum_{i=1}^{n_i} \frac{C_i}{P_i} \leq 1$ n_i = nb de tâche.

plusieur cœur : $u = \sum_i \frac{C_i}{P_i} \leq N_c$ N_c : nb de cœur identique.

Architecture multicores : exercices de révision

Exercice 1 : Analyse d'une architecture

Considérons une architecture de core de processeur capable de réaliser en parallèle :
2 opérations sur des entiers, 1 multiplieur/div, 2 opération sur des flottants, 4 accès mémoire.
1.1 - Quel est le nombre maximum d'instructions qui peuvent être exécutées en parallèle ?
1.2 – La loi de consommation de courant pour cette architecture est :
I(mA)=I_0 + i*I_1 + j*I_2 + k*I_3 + l*I_4
i : nombre d'opération sur les entiers,
j : nombre d'opération de type MULT/DIV
k : nombre d'opération sur les flottants
l : nombre d'accès mémoire
Quelle est l'intervalle de variation de i, j, k, l. Expliquer ce que représentent I_0, I_1, I_2, I_3 et I_4.
1.3 - Quelle campagne de mesure pourrait-on mettre en place pour caractériser cette architecture en consommation ?

2 : opérations entier
1 : multiplieur / div
2 : opération flottant.
4 : accès mémoire.

1.1 : on additionne : 2+1+2+4 ⇒ 9
on obtient 9 instructions en parallèle.

1.2 : la loi de consommation :

$$I(mA) = I_0 + iI_1 + jI_2 + kI_3 + lI_4$$

variation : i → $0 < i < 2$
j → $0 < j < 1$
k → $0 < k < 2$
l → $0 < l < 4$

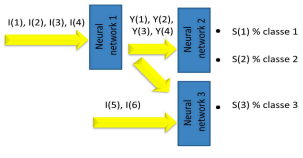
représentant : I_0 = consommation de base du processeur.
I_1 : consommation opération sur les entier
I_2 : consommation multiplieur / div.
I_3 : consommation opération flotant
I_4 : consommation des mémoire.

$$I = I_0 + 2I_1 + I_2 + 2I_3 + 4I_4$$

1.3. Par caractérisation de l'architecture : on réalise une campagne de mesure. en activant séparément chaque type unité.
on mesure d'abord I_0.
puis $I_0 + I_1$
$I_0 + I_2$
$I_0 + I_3$
$I_0 + I_4$
ainsi on détermine I_1, I_2, I_3, I_4.

Exercice 2 : Performance, parallélisme & consommation

Considérons une architecture de core de processeur superscalaire fonctionnant à une fréquence f=1Ghz et offrant le parallélisme suivant : 2 opérations sur les entiers, 2 opérations sur les flottants et 3 accès mémoire.
L'application considérée est un réseau de neurones pour un problème de classification. Ce réseau possède 8 entrées (valeur des caractéristiques), 2 couches et 3 sorties (classe) organisés comme le schéma ci-après.



Une première écriture de cette application est donnée ci-après :

// Neural Network 1

for i **in range** 1 **to** 4 :
Y(i)=W1(i,1)*I(1) + W1(i,2)*I(2) + W1(i,3)*I(3) + W1(i,4)*I(4)

// Neural Network 2

for i **in range** 1 **to** 2 :
S(i)= W2(i,1)*Y(1)+ W2(i,2)*Y(2)+W2(i,3)*Y(3) + W2(i,4)*X(4)

// Neural Network 3:

S(3)= W4(1)*Y(1) + W4(2)*Y(3)+ W4(3)*I(5) + W4(4)*I(6)

2.1 - Déterminer le temps d'exécution idéal pour cette architecture. Préciser les instructions qui seront exécutées en parallèle.
2.2 – Une nouvelle écriture de cet algorithme permettrait-elle d'améliorer le temps d'exécution ? Dans le cas positif, expliquer les gains attendus.
2.3 – Serait-il possible de faire du multiflot ? Si la réponse est oui, comment et quelle décomposition en terme de process serait proposée ? Et quels sont les gains attendus ?
2.4 – Comparer les résultats obtenus pour ces deux architectures et commentez.
2.5 - On donne, ci-après la puissance consommée, en fonction du nombre d'instructions exécutées en parallèle. Calculer l'énergie consommée par l'application décrite ci-dessus avec Fcpu=1Ghz.

2.1 : réseau x 1 :

$$Y(i) = W_0(i,1) \times I(1) + W1(i,2) \times I(2) + W1(i,3) \times I(3) + W1(i,4) \times I(4).$$

3 add
4 mult.
⇒ (3+4) 4 ⇒ 28 cycle → sans parallèle

avec parallélisme : cycle : 2 multi flotant.

cycle		
1	P_1	P_2
2	P_3	P_4
3	P_1+P_2	P_3+P_4
4	$(P_1+P_2)+(P_3+P_4)$	–

hypothèse : les données nécessaires aux calculs (poids w, entrée I, puis résultats intermédiaires) sont déjà chargés dans des registres / mémoire et les accès mémoire ne sont pas comptabilisés dans le temps de calcul.

(pour i=1→4) 4 → 4 cycle → 16 cycle total.

réseaux 2 :

$S(i) = \omega_2(i,1) \times Y(1) + \omega_2(i,2) \times Y(2) + \omega_2(i,3) \times Y(3) + \omega_2(i,4) \times Y(4)$

C_1 : P_1 P_2

C_2 : P_3 P_4

C_3 : P_1+P_2 P_3+P_4

C_4 : $(P_1+P_2)+(P_3+P_4)$ –

(pour i=1→2) 2 → 4 cycle → 8 cycle total :

réseaux 1 :

$S(3) = \omega_4(1) \times Y(1) + \omega_4(2) \times Y(2) + \omega_4(3) \times I(5) + \omega_4(4) \times I(6)$.

C_1 : P_1 P_2

C_2 : P_3 P_4

C_3 : P_1+P_2 P_3+P_4

C_4 : $(P_1+P_2)+(P_3+P_4)$ –

4 cycles :

t_{cycle} total : $t_{tot} = t_{cycle}$ avec : 1 Gy :

$= 16+8+4$ $\Rightarrow \frac{1}{10^9} = 1$ ns.

$= 28 \times \frac{1}{10^9}$

$= 28$ ns.

2.2 · oui, on peut réécrire l'algorithme en différant les boucles afin d'exposer la parallélisme entre les différents $Y(i)$, puis entre les calculs des $S(i)$.
Les $Y(1), Y(2), Y(3), Y(4)$ sont indépendants entre eux et peuvent donc être ordonnancés de manière à mieux utiliser les 2 unités flottantes.
Une fois $Y(i)$ nécessaire calculé, le calcul des réseaux 2/3 peuvent également être exécutés indépendamment
Cette nouvelle écriture permet de réduire le temps d'exécution en améliorant l'utilisation des unités parallèles.

2.3 on décompose :

P_1 : Réseau 1 P_3 : Réseau 3.

P_2 : Réseau 2

le réseau 1 doit d'abord produire les $Y(i)$, ensuite, les réseaux 2,3 étant pas de dépendance entre.

$P_A \rightarrow \{ P_2 \; P_3$ (//)

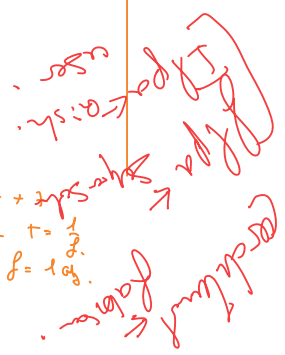
ainsi, le multi-flot est possible · on définit 3 programmes : P_1, P_2, P_3

quand P_1 est fini les P_2, P_3 peuvent être exécuté en réseau 1 → réseau 2 → réseau 3.
parallèle.
Le gain obtenu est une diminution du temps d'exécution.

2.4. $Y(i) = \underbrace{\omega_1(i,1) \times I(1)}_{\text{addr } L_1 \; L_2} + \underbrace{\omega_1(i,2) \times I(2)}_{\text{addr } L_3 \; L_4} + \underbrace{\omega_1(i,3) \times I(3)}_{\text{addr } L_5 \; L_6} + \underbrace{\omega_1(i,4) \times I(4)}_{\text{addr } L_7 \; L_8}$.

	mem	mem	mem	mul	mul
C_1	L_1	L_2	L_3	–	–
C_2	L_4	L_5	L_6	A_1	–
C_3	L_7	L_8		A_2	M_3
C_4				A_4	a_1
C_5				a_2	
C_6				a_3	
C_7	stop.				

Cas avec pipeline



réseau 1 : 4 $t_{tot} = 4 \times 7 + 2 \times 7 + 7 = 49$ ns avec $\tau = \frac{1}{f}$

réseau 2 : 2 $f = 1$ Gy.

réseau 3 : 1

2.5 : consommation avec : $f = 1$ Gy :

C_1	3 → 4	140
C_2	4 → 5	160
C_3	4 → 5	160
C_4	2 → 3	120
C_5	1 → 2	100
C_6	1 → 2	100
C_7	1 → 2	100
	I_0	

$I = 10^{-9} \times (140 + 2 \times 160 + 120 + 3 \times 100) \; 10^{-3}$

$\tau = 1$ ns $I = 0{,}68$ nJ.

$\tau = \frac{1}{f}$